GOTERA — Premium Ethiopian Food

WhatsApp Business Integration Plan

Connecting gotera to the WhatsApp Business Platform: ordering, notifications, an AI assistant and an admin inbox.

Prepared	23 October 2026
Codebase	SvelteKit 2 + Svelte 5, adapter-node, MySQL / Drizzle, Better Auth, Stripe, web-push, nodemailer, Paraglide (en / am)
Scope	Architecture, features, data model, phased roadmap, costs, compliance, risks

Summary. Use Meta's **WhatsApp Cloud API** directly from the existing SvelteKit server. There's no need for a separate bot server. Start with transactional messages (order confirmations, delivery reminders), which pay off quickly and carry little risk. Then add conversational ordering (catalogue browsing, then a Stripe Checkout link sent in the chat). Last, add a Claude-powered assistant in English and Amharic. It can answer questions and manage subscriptions through tools that reuse the logic already in `src/lib/server`.

1. Why WhatsApp for gotera

gotera's customers are mostly in London's Ethiopian and Eritrean community, and in that community WhatsApp is the main messaging app. Today the site reaches customers by email (nodemailer) and opt-in web push. Both are easy to miss, and web push only works for people who installed the PWA or granted permission. Customers tend to open and reply to WhatsApp messages, and the app supports two-way chat, product cards and structured forms.

Goal	How WhatsApp helps	Metric to track
Fewer missed deliveries	Reminder 2 days before delivery with Skip / Change address buttons	Failed-delivery rate
More add-on revenue	“Add berbere to Friday's box?” sent before the cutoff, with one-tap add	Add-on attach rate
Lower churn	Pause / skip offered in chat instead of cancel	Monthly churn
New customers	Click-to-WhatsApp ads, QR on packaging, “Chat to order” button on site	Chat → checkout conversion
Less support load	AI assistant answers FAQ, allergens, delivery areas, order status 24/7	% chats resolved without staff

2. Integration options

Option	Pros	Cons	Verdict
WhatsApp Cloud API (Meta, direct)	No middleman fee, full feature set (templates, catalogues, Flows, interactive messages), free hosting by Meta	We build the webhook, inbox and sending logic ourselves	**Recommended**: the stack already has webhook, cron and dashboard patterns
BSP (Twilio, 360dialog, WATI, Respond.io)	Ready-made inbox UI, quicker setup	Monthly fees and per-message markup, less control, another vendor	Fallback if no dev time
WhatsApp Business app only	Free, zero code	No automation, no API, manual	Keep for day-to-day use via *coexistence*
wa.me links only	Five minutes to add	One-way: opens a chat, nothing automated	Do it now as Phase 0

Coexistence: Meta lets one number be used in both the WhatsApp Business app (on staff phones) and the Cloud API, so the team can keep chatting from their phones while the site automates. Check that this is enabled during onboarding. It is not available in every region or for every number type.

3. Account setup (non-code prerequisites)

- Create or confirm a **Meta Business Portfolio** (business.facebook.com) for GOTERA and complete **business verification** (Companies House number, utility bill or bank statement, website domain). Without verification you have tight messaging limits.
- Create a **Meta developer app** of type Business and add the **WhatsApp** product. This creates a WhatsApp Business Account (WABA).
- Add a **dedicated UK phone number**. It can't currently be registered on personal WhatsApp unless it's migrated or added through coexistence. Get the **display name** “GOTERA” approved.
- Create a **System User** with a permanent access token (scopes `whatsapp_business_messaging`, `whatsapp_business_management`, plus `catalog_management` for commerce).
- Add a payment method to the WABA. Template messages are billed by Meta.

- Write the **message templates** (Section 6) in English and Amharic and submit them for approval. Approval usually takes minutes to hours.
- Optional: build a **Commerce Manager catalogue** and link it to the WABA for product messages.

New environment variables (add to `.env` / server config, read via `$env/static/private`):

```
WHATSAPP_TOKEN=            # System User permanent token
WHATSAPP_PHONE_NUMBER_ID=  # sender id from the WhatsApp > API Setup page
WHATSAPP_WABA_ID=
WHATSAPP_APP_SECRET=       # verifies X-Hub-Signature-256 on webhooks
WHATSAPP_VERIFY_TOKEN=     # random string for the GET handshake
WHATSAPP_CATALOG_ID=       # optional, Phase 2
ANTHROPIC_API_KEY=         # Phase 3
```

4. Architecture in this codebase

Everything runs inside the existing adapter-node server. The integration follows patterns the code already uses. The Stripe webhook (`src/routes/api/stripe/webhook`) is the model for verifying signatures and processing idempotently. The cron endpoint (`src/routes/api/cron/upcoming-deliveries`) is the model for scheduled sends.

```
Customer WhatsApp
      |  (messages, button taps, cart orders, Flow submissions)
      v
Meta Cloud API --webhook--> /api/whatsapp/webhook  (verify signature, store, ack 200 fast)
      ^                              |
      |                              v
      |                    $lib/server/whatsapp/router.ts
      |                     |-- keyword / button handlers (skip, add-on, status)
      |                     |-- commerce: order -> Stripe Checkout link
      |                     |-- agent.ts (Claude + tools)  --> db / stripe / plans / stock
      |                     '-- human handoff -> /dashboard/whatsapp inbox
      |                              |
      '------ graph.facebook.com <---- $lib/server/whatsapp/client.ts (send text / template /
                                        interactive / product messages)
Triggers that SEND:  Stripe webhook (paid, failed)  |  cron (reminders, cutoff nudges)
                     dashboard (delivery dispatched, bulk campaign)
```

New files

Path	Purpose
`src/lib/server/whatsapp/client.ts`	Thin fetch wrapper for `POST /v{N}/{PHONE_NUMBER_ID}/messages`: sendText, sendTemplate, sendButtons, sendList, sendProduct(s), markRead. Retries on 429/5xx.
`src/lib/server/whatsapp/templates.ts`	Typed helpers per approved template (name, language en/am, parameters), similar to `emailTemplates.ts`.
`src/routes/api/whatsapp/webhook/+server.ts`	GET: hub.challenge handshake. POST: verify `X-Hub-Signature-256` with the app secret, dedupe by message id, persist, enqueue/handle, return 200 within a few seconds.
`src/lib/server/whatsapp/router.ts`	Decides what an inbound message is: button reply, list reply, cart order, Flow reply, or free text (passed to the agent).
`src/lib/server/whatsapp/agent.ts`	Claude tool-use loop (Section 7).
`src/lib/server/phone.ts`	Normalise `subscribers.phone` / `addresses.phone` to E.164 (+44...). Use libphonenumber-js.
`src/routes/dashboard/whatsapp/`	Admin inbox: conversations, take over from bot, reply, see linked subscriber/orders.
`src/routes/account/details` (edit)	WhatsApp opt-in toggle + number verification.

Data model additions (Drizzle, `schema.ts`)

```
whatsapp_contacts     id, wa_id (E.164, unique), subscriber_id -> subscribers (nullable),
                      profile_name, language ('en'|'am'), opted_in_at, opted_out_at,
                      last_inbound_at   // drives the 24h free-reply window
whatsapp_messages     id, contact_id, wamid (unique, for dedupe + status), direction (in|out),
                      type, body_json, template_name, status (sent|delivered|read|failed),
                      error_code, created_at
whatsapp_conversations id, contact_id, mode ('bot'|'human'|'closed'), assigned_user_id,
                      summary, updated_at
whatsapp_carts        id, contact_id, items_json, stripe_checkout_session_id, status
subscribers (+)       whatsapp_opt_in boolean, whatsapp_opt_in_at timestamp
```

Long-running work (LLM calls, Stripe calls) should not block the webhook response. Acknowledge immediately, then process. On a single node process a simple in-process queue works, or a `whatsapp_jobs` table drained by the existing cron runner. Meta retries webhooks that don't get a 200, so idempotency on `wamid` is required.

5. What we can do with it

5.1 Transactional notifications (Phase 1)

Event	Trigger in code	Message
Subscription / order paid	Stripe webhook `checkout.session.completed`, `invoice.paid`	Thanks + plan, quantity, first delivery date
Upcoming delivery	cron `upcoming-deliveries` (REMINDER_LEAD_DAYS = 2)	Date + address + buttons *Skip* / *Add extras* / *All good*
Add-on cutoff	same cron, `cutoffDateFor()`	"Last chance to add extras before {cutoff}" with the add-on token link
Out for delivery	dashboard deliveries/route when the driver starts	ETA window, driver contact
Delivered / failed	delivery status change	Confirmation or reschedule options
Payment failed	Stripe `invoice.payment_failed`	Link to update card (Stripe billing portal)
Gift order	giftOrders fulfilment	Message to the recipient (only if they opted in) + to the sender
Partner applications	partnerApplications status	Update to restaurant / shop partners

Reply buttons map straight onto existing server actions: *Skip* writes `delivery_skip_dates`, *Add extras* sends the add-ons list, and *Change address* sends a WhatsApp Flow form or a signed link to `/account/delivery`.

5.2 Buying through WhatsApp (Phase 2)

Important for the UK: native in-chat payments (WhatsApp Pay / payment request messages) are only offered in a few markets such as India, Brazil and Singapore. They are **not available in the UK**, so in the UK we collect payment with **Stripe**, the checkout we already run.

Three ways to sell in the chat, in order of effort:

- **A. Link-out (fastest).** The bot or staff sends an interactive message with a CTA URL button that opens a pre-filled Stripe Checkout Session. This reuses the `stripe.checkout.sessions.create` code in `src/routes/subscribe/+page.server.ts` for both `mode:'payment'` (one-off/guest/gift) and `mode:'subscription'`. Put `wa_id` in session metadata so the Stripe webhook can send a WhatsApp receipt.
- **B. Native catalogue + cart.** Sync plans and sellable add-ons (`sellablePlan()`, `sellableWithInjera`, the `stock` table) to a Meta catalogue via the Catalog API, either on a schedule or when the dashboard edits a product. Customers browse *single-product* and *multi-product* messages and send a cart. The cart arrives as an `order` webhook, which we validate (price, stock, postcode via `/api/postcode`) and answer with a Stripe link.
- **C. WhatsApp Flows.** Multi-screen native forms inside WhatsApp: choose plan, quantity, delivery day, postcode check, address. The Flow posts the result to our endpoint, and we create the subscription checkout. This is the closest to a native "subscribe in WhatsApp" experience.

> **Example flow:** Customer taps the "Order on WhatsApp" ad → bot greets in their language → sends plan carousel / multi-product message → customer sends cart (2× Family Injera + Berbere) → we check postcode and stock → send "Pay £XX securely" button (Stripe Checkout) → Stripe webhook fires → WhatsApp receipt with delivery date → reminder 2 days before → "Delivered — how was it?" review request.

Other commerce ideas: re-order with one reply ("Same as last month?"); **honey orders** and seasonal drops announced to opted-in customers; **referrals**, where we send the customer's referral link so they can forward it to

friends on WhatsApp (this works well because people share through WhatsApp anyway); gift cards sent as a message to the recipient; abandoned-checkout recovery for customers who opted in and gave their number (`cleanupAbandonedCheckouts` already finds them).

5.3 Self-service account management

- "When is my next delivery?" / "skip next week" / "pause for a month" / "change to 2 packs", backed by the same logic as `/account/delivery`, `/account/change-plan` and `/account/cancel`.
- Order history, receipts (Stripe invoice PDF link), and allergen info from `/allergens`.
- Destructive actions (cancel, plan change) always ask for confirmation with a button. Don't act on free text alone.

5.4 Admin and operations

- **Shared inbox** at `/dashboard/whatsapp`. Each conversation shows the matched subscriber, their subscriptions and deliveries. Staff can take over from the bot and hand back. Reuse the dashboard's role checks.
- **Campaigns**: extend `bulkEmail.ts` so it can also send an approved marketing template to opted-in contacts, with segment filters (plan, postcode area, last order date).
- **Driver messages**: from `dashboard/deliveries/route`, send "I'm 10 minutes away" with one tap. Customers can send a location pin or photo back.
- **Internal alerts**: low stock (`stock.ts`), new partner application, failed payments, sent to the owners' WhatsApp group or number through a utility template.
- **Reports**: messages sent/delivered/read, chat-to-checkout conversion, bot resolution rate, added to `dashboard/reports`.

6. Rules to design around

Rule	Implication
24-hour customer service window	After the customer messages us, we can reply freely (any message type) for 24 hours. Outside that window we may only send **pre-approved templates**. Track `last_inbound_at` and have `client.ts` pick free-form or template automatically.
Template categories	*Utility* (order/delivery updates), *Marketing* (promos, re-engagement), *Authentication* (OTP). Meta can recategorise a template, so keep utility templates strictly transactional.
Opt-in required	Explicit consent to receive WhatsApp messages, collected at checkout/signup/account page and stored with a timestamp. Separate from email `marketingOptIn`. Honour "STOP" instantly. UK PECR and GDPR also apply.
Quality rating and limits	New numbers start with a daily cap on business-initiated conversations, which rises with good quality. Blocks and reports lower the rating, so don't over-message.
AI policy	Meta's WhatsApp Business terms (from January 2026) **ban general-purpose AI chatbots** on the API. Assistants that serve the business's own customers for its own products and support are allowed. Keep the agent scoped to gotera.

Starter templates (submit in en + am)

Name	Category	Body (variables in {{n}})	Buttons
order_confirmed	Utility	Thanks {{1}}! Your {{2}} is confirmed. First delivery: {{3}}.	View order (URL)
delivery_reminder	Utility	Your injera arrives {{1}} at {{2}}. Need a change?	All good / Skip / Add extras

Name	Category	Body (variables in {{n}})	Buttons
addon_cutoff	Utility*	Extras for {{1}} close at {{2}}.	Add extras (URL)
out_for_delivery	Utility	Your driver is on the way, ETA {{1}}.	—
payment_failed	Utility	We couldn't take payment for {{1}}. Update your card to keep deliveries coming.	Update card (URL)
new_drop	Marketing	Fresh {{1}} is back this week. Add it to your next box?	Shop now / Stop promos
login_code	Authentication	Your GOTERA code is {{1}}.	Copy code

* Meta may classify cutoff nudges as marketing. Word them as a service notice about an existing order.

7. LLM connection: the gotera assistant

Free-text messages go to an agent built on the **Claude API** (Anthropic SDK, `@anthropic-ai/sdk`) using **tool use**. The model does not touch the database directly. It can only call a small set of typed tools that wrap existing server functions, and every tool re-checks that the WhatsApp contact owns the data it touches.

Model choice

- **Claude Haiku 4.5** (`claude-haiku-4-5`) for routine chat: fast and cheap, and good enough for FAQ, status and skip requests.
- **Claude Sonnet 5.5** (`claude-sonnet-5-5`) for harder conversations, complaint handling, or better Amharic quality. Route based on detected language or escalate after a failed attempt.
- Use **prompt caching** on the system prompt + product/FAQ context (large and the same on every call) to cut cost and latency.

Tools exposed to the model

Tool	Wraps	Notes
get_catalogue	`sellablePlan()`, addonCatalogue, stock	Current prices from DB, never from the prompt
check_postcode	`/api/postcode`, geocode.ts	Delivery area + available days
get_my_account	subscribers, subscriptions, deliveries	Only for a contact linked to a subscriber
skip_delivery / pause	deliverySchedule.ts, delivery_skip_dates	Respects cutoff; confirmation button before executing
add_addon_to_delivery	delivery_addons + addon token flow	Stock-checked
create_checkout_link	Stripe Checkout (subscribe page logic)	Returns URL, sent as a CTA button
get_faq / allergens	FAQ + allergens content (en/am)	Grounding, reduces hallucination
handoff_to_human	whatsapp_conversations.mode = 'human'	Notifies staff (push / WhatsApp alert)

Sketch

```
// src/lib/server/whatsapp/agent.ts  (sketch)
import Anthropic from '@anthropic-ai/sdk';
const client = new Anthropic({ apiKey: ANTHROPIC_API_KEY });

export async function reply(contact, history) {
  const messages = history.map(toClaudeMessage);           // last ~20 turns + summary
  for (let i = 0; i < 6; i++) {                               // bounded tool loop
    const res = await client.messages.create({
      model: 'claude-haiku-4-5', max_tokens: 800,
      system: [{ type: 'text', text: SYSTEM_PROMPT, cache_control: { type: 'ephemeral' } }],
      tools: TOOLS, messages
    });
    if (res.stop_reason !== 'tool_use') return textOf(res);   // -> whatsapp.sendText
    messages.push({ role: 'assistant', content: res.content });
    messages.push({ role: 'user', content: await runTools(res.content, contact) });
  }
  return handoff(contact);
}
```

Guardrails

- System prompt: only gotera topics, answer in the customer's language (English / Amharic), never invent prices or dates, and use tools for anything account-specific. Politely decline unrelated requests (required by the Meta AI policy).

- Authorisation lives in the tools, not the prompt. The phone number links to a subscriber only after verification (OTP template or a signed link from the logged-in account page).
- Money-moving or destructive actions need an explicit button confirmation from the customer.
- Hand off to a human on: anger/complaint detected, 2 failed tool attempts, the customer asks for a person, allergy/medical questions beyond the published list.
- Log every tool call. Review a sample weekly. Turn the bot off per conversation from the inbox.
- Minimise personal data sent to the LLM (no card data ever; addresses only when needed).

More AI uses

- **Voice notes**: many customers prefer voice messages. Transcribe the audio (a speech-to-text service), then pass it to the agent.
- **Photo intake**: customer sends a photo of damaged packaging. Claude's vision classifies it and opens a refund/replacement ticket for staff.
- **Staff copilot** in the inbox: suggested replies, conversation summaries, auto-tagging (delivery issue, billing, sales lead).
- **Insights**: weekly summary of common questions and complaints for the owners, generated from `whatsapp_messages`.
- **Campaign copy**: draft bilingual marketing template text for approval.
- **MCP server (later)**: expose read-only gotera tools (orders, stock, reports) so the team can query the business from Claude.

8. Phased roadmap

Phase	Deliverables	Effort*
0. Quick wins	“Chat with us” floating button + wa.me link on site, FAQ, footer and order emails; QR code on packaging/inserts; WhatsApp Business app profile (hours, catalogue, away message)	0.5 day
1. Foundations + notifications	Meta verification, number, token; webhook endpoint with signature check; client.ts; schema + migration; phone normalisation; opt-in at checkout and account page; templates order_confirmed, delivery_reminder, payment_failed, out_for_delivery; hook into Stripe webhook and cron; Skip / All good buttons; STOP handling	1–2 weeks
2. Commerce	Stripe Checkout link flow with wa_id metadata; catalogue sync; cart order handling; add-on upsell before cutoff; re-order; abandoned checkout recovery; optional WhatsApp Flow for subscribe/address	1–2 weeks
3. AI assistant	agent.ts with tools, en/am prompt, caching, handoff; phone ↔ account verification; evaluation set of ~50 real questions; staged rollout (staff → 10% → all)	1–2 weeks
4. Inbox + growth	Dashboard inbox and takeover; campaigns via bulk email module; reports; Click-to-WhatsApp ads; voice notes and photos; staff copilot	2+ weeks

* Rough single-developer estimates. Meta business verification can take from a day to a few weeks, so start it immediately.

9. Costs (indicative)

Item	Cost model
WhatsApp Cloud API hosting	Free (Meta-hosted)
Service replies (customer messaged first, within 24h)	Free
Utility templates	Per message, a few pence in the UK; **free when sent inside an open 24h window**
Marketing templates	Per message, the most expensive category, so send them sparingly and only to engaged, opted-in customers
Authentication templates	Per message, low
Claude API	Per token. With Haiku 4.5 and prompt caching, a typical support chat costs well under a penny to a few pence
BSP (if chosen instead)	Monthly subscription + markup

Meta moved to **per-message pricing** in July 2025, with rates by category and country. Check the current UK rate card in WhatsApp Manager before budgeting. Example: 300 subscribers × (1 reminder + 1 confirmation) per month is about 600 utility messages. Many will be free because customers reply and open the window.

10. Risks and mitigations

Risk	Mitigation
Number blocked / quality drops from over-messaging	Strict opt-in, frequency cap per contact, easy STOP, mostly utility messages
Template rejected or recategorised	Keep utility wording strictly transactional; prepare fallbacks; email stays as backup channel
Bot gives wrong price, date or allergen info	Data comes only from tools/DB; allergen answers quote the published list; handoff on doubt
Account takeover via phone number	Link number to account only after OTP/signed-link verification; confirm destructive actions

Risk	Mitigation
Webhook duplicates / Meta retries	Dedupe on wamid; idempotent handlers (same pattern as the Stripe webhook)
Amharic quality	Human-reviewed templates; evaluate model output with native speakers before launch; route to Sonnet if needed
Vendor / policy changes	Keep all WhatsApp code behind `$lib/server/whatsapp`; email + push remain

11. Next steps

- 1. Start Meta Business verification and choose the WhatsApp number **this week**.
- 2. Ship Phase 0 (chat button, QR on inserts).
- 3. Draft and submit the 5 utility templates in English and Amharic.
- 4. Build Phase 1 behind a feature flag; test with staff numbers via the Meta test number.
- 5. Collect real customer questions from the first month of chats to build the AI evaluation set for Phase 3.